LOCUS AI

AI-Based Indoor Positioning with Sparse Infrastructure and Noisy Measurements

Team Name: Byte Benders
Hackathon: VIT Code_Apex 3.0
Problem Statement: AI-Based Indoor Positioning with Sparse Infrastructure and Noisy Measurements
Project Type: Software-based simulation, AI, and intelligent location estimation
Hardware Requirement: No physical hardware is required for the proposed prototype.

---

1. The Idea in Simple Words

Imagine a person walking inside a hospital, warehouse, shopping mall, or large office building.

Outside, a mobile phone can often estimate its location using GPS. But inside a building, GPS signals can become weak or unreliable because roofs, walls, and other obstacles interfere with them.

So, how can we estimate where a person is inside a building without relying on GPS?

One possible solution is to install fixed reference points at known locations throughout the building. These reference points are called **anchors**.

A moving person or object receives measurements from these anchors. The measurements help us estimate the person's position.

However, there is a problem: real-world measurements are not always accurate.

Some anchors might provide incorrect readings because of walls or interference. Some anchors might temporarily stop providing readings. There might also be very few anchors available.

Our project, **LOCUS AI**, aims to solve this problem.

We will build a software application that creates a virtual building, simulates anchors and a moving person, generates realistic measurement errors, and uses intelligent algorithms to estimate the person's position as accurately and reliably as possible.

The application will also show which measurements appear unreliable, how confident the system is in its estimate, and how the estimated position changes when the environment becomes more difficult.

In one sentence: LOCUS AI is a virtual indoor-positioning system that learns to make the best possible location estimate from a small number of imperfect measurements.

2. What Problem Are We Solving?

2.1 Understanding the indoor-positioning problem

Let's use a hospital as an example.

A hospital may want to know the approximate location of a staff member, a mobile medical trolley, or a piece of equipment inside its building.

To estimate the location, the system could use fixed anchors installed at known positions. The moving object receives measurements from the anchors, and a computer uses those measurements to calculate its probable location.

For example, suppose a room has four anchors:

- Anchor A is near the top-left corner.
- Anchor B is near the top-right corner.
- Anchor C is near the bottom-left corner.
- Anchor D is near the bottom-right corner.

The anchors have known coordinates on a map. A moving object receives distance estimates from some of these anchors.

The computer combines the measurements to estimate the object's coordinates, such as X = 6.2 metres and Y = 4.8 metres.

That sounds straightforward, but real environments introduce several challenges.

2.2 Challenge 1: Too few anchors

Installing a large number of anchors may cost more money, require more maintenance, and complicate deployment.

What happens if a room has only three or four usable anchors instead of many?

With fewer measurements, the system has less information to determine where the object is and to detect which readings are wrong.

LOCUS AI will test how well different algorithms work when the number of active anchors is reduced.

2.3 Challenge 2: Noisy measurements

A measurement is called noisy when it differs from the true value.

Suppose an anchor is actually 5 metres away from an object. Because of interference or measurement error, it might report a distance of 5.4 metres.

Another anchor might report 4.7 metres.

The system does not automatically know which reading is closest to the truth.

If the algorithm trusts all readings equally, these errors can shift its location estimate away from the correct position.

LOCUS AI will simulate these errors and test methods that can reduce their impact.

2.4 Challenge 3: Walls and obstacles

Suppose an anchor and the moving object are separated by a wall.

Radio signals may travel through, around, or reflect off nearby surfaces. In some situations, the measured distance becomes greater than the actual straight-line distance.

This is one example of a **Non-Line-of-Sight measurement**, commonly called NLOS.

The measurement is not necessarily completely useless, but it may be less reliable than a measurement from an unobstructed path.

LOCUS AI will simulate this situation by introducing extra distance error for selected measurements. If a floor plan is available, the system can also check whether a wall lies between the anchor and the estimated object position.

Importantly, a wall crossing is only evidence that a reading might be unreliable. It does not prove that the reading is wrong.

2.5 Challenge 4: Missing measurements

Sometimes an anchor may not provide a reading at a particular moment.

For example, five anchors may be installed, but only three may successfully report measurements during one update.

An effective positioning algorithm should not automatically stop working just because one or two readings disappear.

LOCUS AI will simulate missing readings and test whether it can still estimate the location from the measurements that remain, along with available movement history.

2.6 Challenge 5: Not knowing how reliable the estimated position is

Suppose the system estimates that an object is at X = 6 metres and Y = 4 metres.

Is that estimate highly reliable? Or could the true position be several metres away?

A useful positioning system should communicate not only its estimated position but also how uncertain that estimate is.

LOCUS AI will attempt to estimate this uncertainty and display it on the map. If the available measurements are too weak to support a reliable estimate, the system should be able to flag the result as uncertain instead of pretending that it knows the exact position.

---

3. Our Proposed Solution

LOCUS AI will be a simulation-based application with an interactive dashboard and a location-estimation engine.

It will contain a virtual building, fixed anchors, a moving target, simulated measurements, positioning algorithms, reliability checks, and visual reports.

The entire environment will run in software.

We will not need to purchase sensors, install anchors in an actual building, or track a real person for the hackathon prototype.

The virtual environment will let us create many repeatable experiments. We can change the number of anchors, increase measurement noise, introduce obstacles, and remove readings. We can then observe which algorithms perform better under each condition.

Our central goal is to make location estimation more dependable when the available information is limited or unreliable.

4. What Exactly Will We Build?

The final proposed prototype will have six main parts.

Part 1: Virtual Building Simulator

This will create a digital representation of an indoor environment.

For example, we could simulate a rectangular room measuring 20 metres by 15 metres. The application will display the room on a two-dimensional map.

The user will be able to see:

- The room boundaries.
- The locations of fixed anchors.
- Walls and selected obstacles.
- The true position of a simulated moving object.
- The estimated position calculated by the algorithm.
- The movement path of the object.

We may create several layouts, such as a simple room, a corridor-based layout, and a room with more complicated walls.

The simulator is important because it gives us a controlled environment in which we know the actual location. This lets us measure how far the estimated location is from the correct one.

Part 2: Anchor and Measurement Simulator

The second part will generate the measurements used by the positioning algorithms.

Each anchor will have a fixed coordinate, such as:

- Anchor A: (0, 0)
- Anchor B: (20, 0)
- Anchor C: (0, 15)
- Anchor D: (20, 15)

These are example coordinates for a 20-by-15-metre room.

The moving object will have a position that changes over time.

The simulator can calculate the true geometric distance between the target and each anchor. It will then add simulated errors to create measurements resembling imperfect real-world readings.

For example:

Anchor	Actual distance	Simulated reading	Interpretation
A	5.0 m	5.2 m	Small error
B	7.0 m	6.8 m	Small error
C	4.0 m	6.7 m	Possible large error
D	8.0 m	Missing	No reading received

These numbers are only an illustration of how the system will work, not measured prototype results.

The algorithm will receive the simulated readings, not the true distances. The true distances and target coordinates will be kept separately so that we can evaluate the estimates without giving the answers to the algorithm.

We will simulate several types of measurement problems:

Normal noise: Small random errors in the measured distance.

NLOS-like bias: Extra error added to selected measurements to represent possible wall-related or obstruction-related problems.

Missing readings: Some anchors fail to provide a measurement during an update.

Changing availability: An anchor may work for several updates and then become temporarily unavailable.

Different anchor densities: The simulator can use different numbers of active anchors, allowing us to test how reduced infrastructure affects performance.

We can also provide an optional RSSI-based mode. RSSI means *Received Signal Strength Indicator*, a measure of how strong a received radio signal is. The system can convert simulated

signal strength into an approximate distance. Because this conversion is uncertain, the resulting distance must be treated as an estimate rather than an exact measurement.

Part 3: Location Estimation Engine

This is the core of the project.

It will take the available anchor coordinates and their reported measurements, then calculate the most likely position of the target.

We will develop the system in stages.

A. Baseline positioning algorithm

First, we will implement a standard method called **least-squares multilateration**.

In simple terms, each anchor provides a distance estimate. A possible target position can be compared with those distances.

The algorithm searches for the position whose predicted distances best match the reported distances.

Imagine several circles drawn around the anchors. Each circle represents a reported distance. In an ideal situation, the circles would meet at the target's location. In a real situation, they may not meet at one exact point because the measurements contain errors.

The algorithm finds a position that provides a reasonable overall fit to the measurements.

This first algorithm will serve as our baseline. It will help us measure whether our more advanced methods actually improve the results.

B. Robust positioning algorithm

The baseline method may be affected by a measurement that is far more inaccurate than the others.

For example, suppose three measurements are approximately correct, but one distance is much larger than the true distance because of a simulated obstruction.

A basic algorithm may be pulled towards that incorrect measurement.

To address this problem, we will investigate a robust estimation method, such as **Huber-loss optimization**.

In simple words, robust estimation attempts to stop unusually large errors from dominating the final position estimate.

The system will not automatically discard every unusual reading. Instead, the estimator can reduce the influence of a reading that appears inconsistent with the others.

We will compare the robust method with the baseline to determine whether it actually improves performance across different conditions.

C. Anchor Trust Engine

Our proposed Anchor Trust Engine will assess how reliable the available measurements appear to be.

Think of it as a system that checks the quality of the clues before using them to determine a location.

It may examine:

- How closely each measurement agrees with the current estimated position.
- Whether the measurement has changed suddenly compared with recent values.
- Whether the anchor has a history of highly variable readings.
- Whether the anchor has frequently been unavailable.
- Whether the floor plan suggests a wall may lie between the anchor and the estimated target position.

These indicators can be combined to assign a reliability score or confidence weight to a measurement.

A high score would indicate that the reading appears more consistent with the available evidence. A low score would indicate that the reading deserves additional caution.

The score will not guarantee that the reading is correct. A consistently biased measurement may sometimes appear reasonable, while a correct reading may occasionally look unusual.

The Trust Engine will therefore be evaluated on simulated scenarios where we know which measurements were deliberately corrupted.

The resulting reliability scores can help the robust estimator decide how strongly to rely on different measurements.

D. Movement tracking and temporal filtering

A moving object does not usually teleport randomly from one side of a room to the other. Its current location is related to where it was a moment ago.

We can use that information to make the estimates more stable.

For example, the system estimates that the target is at position A during one update. During the next update, the measurements suggest a position several metres away. If the target is supposed to be walking slowly, this sudden jump may deserve closer examination.

We will investigate a **Kalman filter**, a mathematical technique that combines new measurements with a prediction based on previous movement.

The filter can smooth random fluctuations and produce a more stable estimated trajectory.

However, movement history should not override strong evidence. If the target genuinely moves quickly or changes direction, the system must be able to adapt.

For a fair real-time demonstration, the filter will use current and past information only. It will not use future measurements to improve an earlier estimate.

E. Confidence and uncertainty estimation

The system should also estimate how uncertain its location estimate is.

For example, two situations might both produce the estimated position (6, 4) metres.

In the first situation, many consistent measurements are available. In the second, only a few unreliable measurements are available.

The position coordinates are identical, but the level of confidence should not be.

We will develop an uncertainty estimate based on the available measurements and the estimation model. Where appropriate, we will display an uncertainty ellipse around the estimated position.

A smaller ellipse generally indicates a more concentrated estimate, while a larger ellipse indicates greater uncertainty.

We will also test whether these uncertainty estimates are meaningful. A stated confidence level should correspond reasonably well to the actual errors observed on held-out simulations.

If the available information is insufficient to support a defensible estimate, the system should be able to flag the result as unreliable or abstain from presenting it as a confident location.

We will not claim that every estimate is correct.

Part 4: Two Operating Modes

A distinctive feature of LOCUS AI will be its two operating modes.

Plan-Free Mode

Plan-Free Mode will operate using information such as:

- The known coordinates of the anchors.
- The measurements received from each anchor.
- The consistency of current and historical readings.
- The movement history of the target, when available.

It will not require a building floor plan.

This mode is important because a floor plan may not be available in every environment. It also provides a way to test how well the algorithms work using only the positioning measurements and their history.

Plan-Aware Mode

Plan-Aware Mode will use the same basic information, but it will also have access to a floor plan.

The floor plan can provide information about walls and room boundaries. The system can examine whether a wall lies between an anchor and a possible target position.

This information can help identify measurements that may be affected by obstructions.

However, the map itself may be incomplete or inaccurate. For that reason, the system should not blindly reject measurements whenever a wall appears between an anchor and the target.

Instead, wall information will be combined with measurement consistency and recent behaviour to assess reliability.

We will compare Plan-Free Mode and Plan-Aware Mode under the same test conditions to determine when the additional map information is genuinely useful.

5. Where Does AI Come Into the Project?

LOCUS AI combines mathematical estimation, reliability assessment, simulation, and potentially a machine-learning model.

It is important to distinguish these parts instead of describing every calculation as AI.

Our initial positioning algorithms, including least-squares estimation and robust optimization, are mathematical methods.

The reliability engine can initially use transparent, rule-based checks. This gives us a working baseline and makes it easier to understand why a measurement was treated as suspicious.

We may then develop a **Random Forest classifier** to classify measurements as likely normal or potentially unreliable.

A Random Forest is a machine-learning method that combines the decisions of multiple decision trees.

For training, the simulator can produce examples where we know whether a measurement was generated normally or deliberately corrupted. This gives the model training labels.

Possible model inputs include:

- Current measurement residual, meaning the difference between the observed distance and the distance predicted by the current position estimate.
- Recent changes in measurement residuals.
- Sudden distance jumps.
- Rolling measurement variability.
- Missing-reading frequency.
- Wall-crossing information when the map is available.

The model would estimate whether a reading resembles the types of unreliable measurements seen during training. Its output could then support the Trust Engine.

We will only include this model in the final system if it performs usefully on independent test scenarios. If a simpler rule-based system performs better, is more reliable, or is easier to validate, we will use the simpler method rather than add machine learning just for appearance.

The critical rule is that the localization algorithm must not receive the hidden true target position or the hidden labels that identify corrupted measurements. Those values will be used only to train models where appropriate and to evaluate the final results.

6. What Will the User Actually See?

We will build an interactive dashboard using Python tools such as Streamlit and Plotly.

The dashboard will be the visible part of our project. A judge or user will be able to see the virtual environment, adjust its conditions, and compare the results.

The proposed interface will include the following sections.

A. Indoor Map

The map will display:

- The boundaries of the virtual building.
- The fixed anchor locations.
- The true simulated target position, when running a simulation.
- The estimated position calculated by LOCUS AI.
- The true and estimated movement paths.
- Walls or obstacles in Plan-Aware Mode.
- An uncertainty ellipse around the estimate, when available.

The true position will be shown in the evaluation view so that the algorithm's accuracy can be assessed. In an operational-style view, the algorithm will not be given that ground-truth position.

B. Anchor Reliability Panel

A separate panel will show the status of the anchors.

For each anchor, the application can display whether a reading is available, the latest measurement, its consistency with the current estimate, and its reliability score.

It may also indicate when a reading is missing or appears suspicious.

This makes the system easier to understand than a basic application that only produces coordinates.

C. Simulation Controls

The user will be able to change experimental conditions, such as:

- The number of active anchors.
- The amount of normal measurement noise.
- The proportion of measurements affected by simulated NLOS-like bias.
- The percentage of missing readings.
- Whether a floor plan is available.
- Whether temporal filtering is enabled.

We will select suitable default values so the system can be demonstrated without requiring the user to understand every technical setting.

D. Break-It Mode

Break-It Mode will allow users to deliberately make the environment more challenging.

For example, a judge could disable two anchors, increase the measurement noise, introduce biased readings, or increase the frequency of missing measurements.

The dashboard will then show how the different positioning methods respond.

This is valuable because the demonstration will not depend only on showing a system working under easy conditions. It will also expose situations where the system struggles.

E. Benchmark and Comparison Panel

The dashboard will compare the performance of different methods, such as:

1. Baseline least-squares multilateration.
2. Robust estimation.
3. Robust estimation with measurement reliability weighting.
4. Robust estimation with causal temporal filtering.
5. Plan-Aware and Plan-Free variants.
6. A machine-learning-assisted variant, if testing shows it is useful.

The comparison will display measured performance rather than assuming that the more complicated algorithm must be the best one.

---

7. How the Complete System Will Work

Here is the intended end-to-end workflow.

Step 1: Create the virtual environment

The simulator generates a room or building layout and places anchors at known coordinates.

Step 2: Generate a moving target

The simulator creates a sequence of positions representing a moving person or object. This sequence provides the true location for later evaluation.

Step 3: Generate measurements

The system calculates the geometric distance between the target and each anchor. It then adds simulated noise, possible bias, and missing readings.

Step 4: Collect the available information

The positioning system receives the anchor coordinates and the available measurements. It is not given the hidden true target coordinates.

Step 5: Estimate the location

The baseline or robust positioning algorithm calculates the most likely target position from the measurements.

Step 6: Assess reliability

The Anchor Trust Engine checks measurement consistency, recent history, availability, and wall information when a floor plan exists.

Step 7: Improve the estimate

The system can use reliability weights and temporal filtering to improve the estimate, provided these methods help during evaluation.

Step 8: Estimate uncertainty

The application estimates how uncertain the result is and flags situations where the evidence is inadequate.

Step 9: Display the result

The dashboard displays the estimated location, the movement path, measurement reliability, uncertainty, and comparison with the true simulated location.

Step 10: Evaluate performance

The evaluation engine records the difference between the true and estimated positions across many test scenarios.

This gives us evidence about where LOCUS AI works well, where it struggles, and which methods are actually useful.

---

8. A Complete Example

Suppose we create a virtual hospital room measuring 20 metres by 15 metres.

We place four fixed anchors around the room. A simulated person is walking through the room and is actually located at coordinates (8, 6) metres.

The simulator generates distance measurements from the four anchors.

Three readings are relatively close to their true distances. However, the fourth reading is affected by a simulated wall-related bias and reports a significantly larger distance.

The system does not know the person's true coordinates. It only sees the anchor coordinates and the measurements.

What would happen with a basic method?

The basic method tries to find the position that best matches all the distances. Because one reading is wrong, its estimate may move away from the actual position.

What would happen with LOCUS AI?

First, the baseline algorithm calculates an initial estimate.

Next, the reliability engine checks how well each measurement agrees with the estimated position and whether the reading has a history of unusual behaviour. In Plan-Aware Mode, it can also check for a possible wall between the anchor and the estimated position.

The robust estimator reduces the influence of measurements that appear inconsistent. If previous positions are available, temporal filtering can help stabilize the result.

The system then displays its estimated location and uncertainty. It also shows which reading appeared suspicious.

Finally, the evaluation module compares the estimated position with the hidden true position at (8, 6) metres.

We repeat the same experiment across many trajectories and different noise conditions to find out whether the robust method improves the results consistently.

This is the fundamental idea behind LOCUS AI: use multiple imperfect clues, evaluate their reliability, and calculate the best-supported location estimate.

---

9. How Will We Measure Whether the Project Works?

A convincing project needs to demonstrate performance, not just show a visually attractive map.

Since we control the simulated environment, we can record the true position separately from the estimate. This allows us to calculate the actual positioning error.

Positioning error

Positioning error is the straight-line distance between the true position and the estimated position.

For a true position (x_t, y_t) and an estimated position (x_e, y_e), the error is:

$$e = \sqrt{(x_e - x_t)^2 + (y_e - y_t)^2}$$

For example, if the true position is (5, 4) metres and the estimate is (5.3, 4.4) metres, the error is:

$$e = \sqrt{(5.3 - 5)^2 + (4.4 - 4)^2} = 0.5 \text{ m}$$

This is an illustrative calculation, not a result from our implementation.

We will report several metrics.

Mean error: The average positioning error across the evaluated samples.

Median error: The middle positioning error after sorting the errors. This helps us understand typical performance without allowing a few very large errors to dominate the summary.

90th-percentile error: The error threshold below which 90% of the evaluated estimates fall. This helps reveal how the system behaves beyond typical cases.

Availability or abstention rate: How frequently the system produces an estimate rather than flagging the available information as insufficient.

Uncertainty calibration: Whether the displayed uncertainty corresponds reasonably well to the errors actually observed.

Processing time: How long the system takes to produce an estimate.

We will also examine performance separately by anchor count, noise level, NLOS-like bias, and missing-reading rate.

Planned testing conditions

We will evaluate the system under multiple combinations of conditions. For example:

Test dimension	Example settings
Active anchors	3, 4, 5, 6
Normal distance-noise standard deviation	0.1 m, 0.3 m, 0.75 m
Measurements with simulated NLOS-	0%, 10%, 25%, 40%

Test dimension	Example settings
like bias	
Missing measurements	0%, 10%, 30%
Floor-plan access	Plan-Free and Plan-Aware
Temporal filtering	Off and on

These are proposed experimental settings. We may adjust them to suit the implementation and computational budget.

We will use the same held-out test scenarios to compare the positioning methods fairly.

We will also separate development data from evaluation data by trajectories and, where feasible, building layouts. This reduces the risk of achieving impressive results simply by testing on scenarios that the algorithms have already seen.

The simulator should be more varied than the estimator's assumptions. For example, test data can include different bias patterns, wall-related errors, and temporal variation. This helps us test whether the system generalizes beyond a perfectly convenient simulation.

We will not claim a particular accuracy improvement until the implemented system has been evaluated and the results have been recorded.

---

10. What Is New or Valuable About Our Approach?

Our project is not simply a map that displays coordinates.

It aims to investigate how an indoor-positioning system can remain useful when its measurements are imperfect and its infrastructure is limited.

The main proposed contributions are:

1. Positioning with sparse infrastructure: We will explicitly test how performance changes when fewer anchors are available.

2. Robust estimation: We will compare ordinary least-squares estimation with an estimator designed to reduce the influence of unusually large errors.

3. Measurement reliability assessment: The Anchor Trust Engine will combine measurement consistency and available history to assess which readings deserve more caution.

4. Two operating modes: Plan-Free Mode works without a floor plan, while Plan-Aware Mode uses available wall information as an additional clue.

5. Movement-aware estimation: We will test whether past motion information makes position estimates more stable without hiding genuine movement.

6. Uncertainty reporting: The system will attempt to communicate how confident the estimate is, rather than returning coordinates without context.

7. Interactive failure testing: Break-It Mode will allow judges to change environmental conditions and observe the response of different algorithms.

8. Transparent evaluation: We will measure performance using known simulated positions and independent test scenarios.

These are proposed contributions, not claims that every feature has already been implemented or that every feature will outperform simpler methods.

11. What Is the Difference Between Our Project and a Real Tracking Product?

This is an important distinction.

A real tracking system would receive actual measurements from physical anchors or radio devices installed in a building.

Our hackathon prototype will generate those measurements in software.

We will build and test the algorithms in a simulated environment before attempting any real-world deployment.

That gives us several advantages: we can create unlimited test trajectories, know the actual target position, deliberately introduce failures, and compare algorithms under repeatable conditions.

However, a simulation cannot fully reproduce every complication of a real building. Its conclusions must therefore be limited to the scenarios and assumptions we test.

The current prototype will **not** locate a real person inside an actual hospital or office building. It will demonstrate how the proposed positioning system estimates location from simulated data.

Physical sensor integration, real radio measurements, and testing in actual buildings would be possible future extensions.

12. Technology We Plan to Use

The proposed implementation stack is Python-based.

Technology	Purpose
Python	Main programming language and application logic
NumPy	Numerical calculations and simulated

Technology	Purpose
	measurements
Pandas	Organizing data and evaluation results
SciPy	Optimization and numerical methods
scikit-learn	Optional Random Forest classifier and model evaluation
Streamlit	Interactive web application and controls
Plotly	Interactive maps, trajectories, and charts
Matplotlib	Evaluation plots and report graphics
pytest	Testing the correctness of core functions
joblib	Saving and loading a trained model, if one is used

The team can work in Visual Studio Code.

No hardware is required for the core prototype. We will generate the inputs in software and process them using the algorithms we implement.

13. Proposed Software Architecture

The application will be divided into modules so that the team can develop and test different parts independently.

Module 1 — Environment Simulator

Creates the virtual floor plan, anchors, target trajectory, and true positions.

Module 2 — Measurement Generator

Produces distance measurements with controllable noise, simulated NLOS-like bias, and missing readings.

Module 3 — Baseline Localizer

Implements ordinary least-squares multilateration as a reference method.

Module 4 — Robust Localizer

Implements robust optimization to reduce sensitivity to inconsistent measurements.

Module 5 — Anchor Trust Engine

Calculates measurement reliability indicators from residuals, trends, variability, availability, and optional wall information.

Module 6 — Temporal Filter

Uses previous estimates and a movement model to smooth the trajectory.

Module 7 — Uncertainty Estimator

Estimates location uncertainty and supports reliability checks.

Module 8 — Evaluation Engine

Calculates positioning metrics and creates performance comparisons across test scenarios.

Module 9 — Interactive Dashboard

Connects the modules and displays the simulation, location estimates, reliability information, and test results.

The modules will share clearly defined data formats. We will also maintain separate ground-truth data for evaluation so that the algorithms cannot accidentally use the answers they are supposed to estimate.

14. How We Will Develop the Project

We should build the project in the right order. Starting with a sophisticated dashboard or machine-learning model before we have a working location estimator would add unnecessary risk.

Stage 1: Build the smallest working simulation

We will create one virtual room, place several anchors, generate a moving target, and produce simulated distance measurements.

At this stage, we only need a basic view of the room and the data generated by the simulator.

Stage 2: Implement baseline positioning

We will implement least-squares multilateration and calculate the estimated target position.

We will compare the estimate with the known simulated position to make sure the algorithm works as intended.

Stage 3: Add difficult measurement conditions

We will introduce noise, biased measurements, and missing readings.

This will give us a way to test how the baseline behaves under increasingly difficult conditions.

Stage 4: Add robust estimation and reliability checks

We will implement robust optimization and develop the first version of the Anchor Trust Engine.

We will then compare the results with the baseline. If a reliability rule causes worse estimates, we will investigate and revise it rather than assuming the new method must be better.

Stage 5: Add movement history and uncertainty

We will investigate causal temporal filtering and a meaningful uncertainty estimate.

We will verify whether the trajectory becomes more stable without making the estimate unresponsive to genuine changes in movement.

Stage 6: Implement Plan-Free and Plan-Aware modes

We will compare the system with and without wall information and test whether the map improves the result.

We will also test imperfect map information, where practical, so that the system does not rely only on a perfectly accurate floor plan.

Stage 7: Build the dashboard

We will connect the working algorithms to Streamlit and Plotly.

The interface will provide controls for simulation settings and display the true path, estimated path, uncertainty, anchor reliability, and performance metrics.

Stage 8: Add machine learning only if justified

Once the system has a working baseline and robust estimator, we will test whether a Random Forest classifier adds value.

We will keep it only if independent evaluation shows a useful improvement.

Stage 9: Test, evaluate, and prepare the demo

We will run repeatable experiments, record the metrics, test difficult scenarios, and prepare a demonstration that explains both strengths and limitations.

15. How We Will Divide Work Among Team Members

The work can be separated into four main responsibilities.

Team coordination and integration: Coordinate the overall architecture, agree on the module interfaces, integrate the code, monitor progress, and prepare the final explanation and presentation.

Simulation and data generation: Build the virtual room, anchors, target trajectories, distance calculations, noise generation, NLOS-like bias, and missing-reading simulation.

Localization and reliability: Implement the baseline and robust estimators, develop the Anchor Trust Engine, explore temporal filtering, and investigate the optional machine-learning component.

Dashboard and evaluation: Develop the interactive interface, visualization, experiment controls, evaluation metrics, charts, tests, and technical documentation.

If fewer people are available for a particular part, related responsibilities can be combined. Everyone should understand the complete workflow even if each person owns a different module.

16. What Could Go Wrong, and How Will We Handle It?

A good technical project should recognize its limitations early.

Problem: Too few anchors provide insufficient information.

In two-dimensional positioning, three suitable distance measurements are generally needed for basic geometric localization, and even then the configuration and measurement quality matter. With too few independent measurements, the position may be ambiguous or unreliable.

Our response will be to test multiple anchor configurations, use available motion history where appropriate, and flag estimates when the information is insufficient.

Problem: The Trust Engine incorrectly identifies a valid reading as unreliable.

A valid reading can disagree with other measurements because of geometry, noise, or a genuine change in movement.

Our response will be to use multiple indicators instead of automatically rejecting a measurement for one reason. We will measure the impact of trust-based weighting and retain simpler methods where they perform better.

Problem: The machine-learning model works on training data but fails on unseen scenarios.

A model may learn patterns specific to the simulated data.

Our response will be to separate training and testing scenarios, vary the simulation conditions, and include the model only if it performs usefully on held-out cases.

Problem: The uncertainty estimate looks convincing but is poorly calibrated.

A small uncertainty ellipse is not useful if the true location frequently falls outside it.

Our response will be to compare uncertainty estimates against actual errors across held-out test cases and report the calibration limitations honestly.

Problem: The floor plan is imperfect.

A missing wall or inaccurate wall location can make the map misleading.

Our response will be to treat map information as evidence, not certainty, and compare Plan-Aware Mode with Plan-Free Mode.

Problem: The project becomes too large to finish in the available time.

Our response will be to prioritize the simulator, baseline localizer, robust estimator, evaluation, and one clear interactive demo. Advanced machine learning and other extensions will come only after the core workflow works.

17. What Will Be Included in Our Final Hackathon Demo?

The core demo will show one virtual target moving through an indoor environment.

At first, we will run the simulation under reasonably clean conditions and show the estimated trajectory.

Next, we will increase the noise or introduce biased readings. We will demonstrate how the baseline estimate changes and compare it with the robust estimator.

Then we will disable one or more anchors to simulate reduced infrastructure.

We will switch between Plan-Free Mode and Plan-Aware Mode to show how the availability of wall information can affect the result.

Finally, we will display the anchor reliability panel, uncertainty estimate, and quantitative comparison across test scenarios.

The judge should be able to understand what the system is doing simply by watching the map and changing the simulation controls.

The demo will also make clear that the system can fail under sufficiently difficult conditions. That is why reliability checks and uncertainty reporting matter.

18. What Success Looks Like

We should define success in terms of a working, testable prototype rather than an unsupported accuracy claim.

The project will be in a good position when:

- The simulator generates reproducible indoor-positioning scenarios.
- A baseline algorithm estimates positions from the available anchor measurements.
- Robust estimation can be compared with the baseline.
- The system handles missing measurements where the available geometry supports a solution.
- The Anchor Trust Engine produces understandable reliability information.
- Plan-Free and Plan-Aware modes can be compared.
- The uncertainty estimate is evaluated against actual simulated errors.
- The dashboard displays the true and estimated trajectories in evaluation mode.
- The evaluation engine reports meaningful metrics under several anchor, noise, and bias conditions.

- The team can explain which parts work, which parts have limitations, and what the test results actually demonstrate.

We do not need to claim that LOCUS AI will always identify the correct location. We need to show that we have designed a sensible method, implemented it correctly, tested it fairly, and learned from the results.

19. Future Possibilities

If the core system works well, possible future extensions include:

Real measurement integration: Feed measurements from actual positioning hardware or radio systems into the software.

More realistic building models: Simulate complex corridors, multiple rooms, different obstruction types, and imperfect floor plans.

Anchor placement optimization: Investigate how to choose anchor positions to balance coverage, cost, and expected positioning accuracy.

Additional signal models: Explore more realistic RSSI, multipath, and correlated measurement errors.

Real-time deployment: Adapt the system for processing live measurements from a physical installation.

These are future possibilities. They are not required to demonstrate the current simulation-based idea.

20. Final Project Summary

LOCUS AI addresses the challenge of estimating indoor location when only a small number of anchors are available and the measurements are noisy, biased, or missing.

We will build a virtual indoor environment where a target moves and fixed anchors provide simulated distance measurements. We will develop a baseline positioning algorithm, compare it with robust estimation, and create an Anchor Trust Engine to assess measurement reliability.

We will investigate temporal filtering to use movement history, uncertainty estimation to communicate how reliable an estimate is, and two operating modes: Plan-Free Mode and Plan-Aware Mode. An optional machine-learning classifier will be included only if it offers measurable value.

A web-based dashboard will allow users to change the simulated environment, introduce failures, compare algorithms, and observe the results on an interactive map. The evaluation engine will measure positioning error and other performance indicators across independent test scenarios.

The prototype will be software-only, allowing us to test many situations without physical hardware. Its purpose is to demonstrate and evaluate a more reliability-aware approach to indoor positioning, rather than claim that it already tracks real people in real buildings.

Our core objective is to estimate indoor location as reliably as possible from limited and imperfect measurements—and to tell the user when the available information is not reliable enough.

A Simple Explanation You Can Say to Anyone

"Imagine you are inside a hospital where GPS does not work properly. We have a few fixed reference points at known locations, and a computer receives estimated distances from those points. But some distances are wrong because of walls or interference, and sometimes an anchor does not provide a reading.

We are building LOCUS AI, a software system that simulates this situation and tries to estimate where a moving person or object is. It checks how consistent the readings are, reduces the influence of suspicious measurements, and uses movement history when helpful. It can also use a floor plan if one is available.

We will show everything on an interactive map, including the estimated position, the movement path, suspicious measurements, and how uncertain the estimate is. We will also compare our algorithm with a basic method to see whether our approach actually improves accuracy under difficult conditions.

We are not building a physical tracking device for this hackathon. We are building and testing the intelligent software behind one."

One-Line Pitch

LOCUS AI is a simulation-based indoor-positioning system that estimates location using sparse anchors, detects potentially unreliable measurements, and communicates uncertainty when indoor conditions become challenging.